

1 Interpreted Java Service2

1.1 Introduction

Interpreted Java services version 2 are web services that are converted by an interpreter to SELECT SQL statements. The result is converted to a web service result, once the SQL statement has been executed.

The definition for the interpreter is created in XML files using the repository client.

1.1 Availability

As of SP7

1.2 Definition

An interpreted Java service is defined in the repository (for further information on the required values and their meaning, refer section "repository data" below).

The service domain can be exported as XML file, once the definition has been completed. The resulting files (the ones relevant to the interpreter) are **<Domain>.Configuration.xml** and **<Domain>.do.xml**.

1.3 Storage in a server

Both files are located on the server at:

JDIR\MOC\<SYSTEM>\listInterpreter\<Scope>.

or JHYDRADIR\MOC\<SYSTEM>\listInterpreter\<Scope>. The scope can have one of the following values: standard, custom or local.

1.4 Available Special Parameters

Each interpreted Java service includes special parameters that are always available and can always be used. Subject to how the interpreted Java service is customized in the file <Domain>.do.xml, the parameters affect processing or are ignored.

These parameters are available:

Name	Data type	Operators	Description
checkresponsibilityarea	boolean	EQUAL	Is only effective if checkRespAreaMode and

			checkRespAreaField are configured. If false, the responsibility area will not be checked. If true, it is checked. The default value is true.
checkresponsibilityareafuctions	String[]	EQUAL, IN	Is only effective, if checkRespAreaMode and checkRespAreaField are configured and checkresponsibilityarea == true. This parameter controls which functions are checked by the responsibility area. The default value is select. The following functions can be indicated: create (vab_tab.anlegen='J') delete (vab_tab.loeschen='J') select (vab_tab.anzeigen='J') update (vab_tab.aendern='J') use (vab_tab.verwenden='J')
longtermdata	boolean	EQUAL	Is only effective if tableClauseLongterm is configured. If false, no long-term data is used. If true, it is checked. The default value is false.

1.5 Repository data

1.5.1 Tab Services

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Name	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Service Function	The function of the service (e.g. list)	
Service Type	The type - for interpreted Java services: fixed InterpretedJavaService2	

Description (German)	Brief (internal) description of the service	
----------------------	---	--

1.5.2 Tab ServiceParameter

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Service Function	The function of the service (e.g. list)	
Service	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Acronym	Acronyms (e.g. person.id) have to be unique for service and result set.	
Web service type	The data type of the parameter (decimal, integer, string, boolean, datetime)	
DB table	The table that is used to select the value for the acronym	
DB field	The field from which the value for the acronym is to be selected This is either just the field name or the expression (if it is a calculated field) including placeholder for the table alias (e.g. <code>hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)</code> or <code>{fn substring(%1\$s.field,2,1)}</code>). The placeholder for the table alias is always <code>"%1\$s"</code> .	
DB Alias	The table alias for the table that is used to select the value for the acronym	
Conversion Method	Here you can specify transformations for input and result parameters (e.g. conversion bool to J/N and vice versa or the correct filtering for datetime fields that consist of two fields in the database). Possible transformations are described elsewhere.	X
Can ...	Have to be set for filter parameters (for Boolean only Can Equal; for string all and for the others, everything except Can Like and Can Like or null)	X (if only Result)

IsFilterParameter	Specifies whether or not the field is a filter field. A filter parameter including its operator is directly converted into an SQL fragment	X (if only Result)
IsResult	Specifies whether or not it is a Result	X (if only Filter)
IsSpecialParameter	Specifies whether or not a field is a special parameter. At the moment, standard processing only supports the above-mentioned special parameters. Further parameters can be used in exits. Special parameters are "options" that cannot directly be converted into an SQL fragment of the type <DB field> <Operator> <Value>.	X
IsMandatory	Specifies whether or not it is a mandatory field. If TRUE and the parameter is missing, an error message is generated at runtime. Is checked with special parameters and filter parameters.	X
InputAsArray	Specifies whether the field also supports arrays as input parameters. (e.g. the operators IN or BETWEEN require an array as input parameter)	
ConditionalFieldKey	This field specifies if a DB field is only conditionally available. The condition as to whether the field is available is checked using the Configuration Manager (see Configuration Manager in the section "server"). The feature key of the Configuration Manager (feature set) has to be entered in this field within the repository for checking.	X
DBFieldAlternative	Only relevant if it is a conditional field. Here you can enter the alternative value. This value can be a figure, null, 'string', {fn ...}, or even another field / subselect. Default value is zero if nothing else is entered. Note: This field has a different behavior than "DB Field". The alias is not automatically put in front. If you want to use the alias of the field "DB Alias", you must put %1\$. in front of the field name.	X
Constraints	If the value "SKIP_INTERPRETER " is entered here, the interpreter ignores this acronym. This is useful, if you want to edit or add acronyms in an exit.	X

1.5.3 Tab Dataobjects

Name	Meaning	Optional
name	Name of the data source. Must be identical to the complete service name, i.e. domain.function (e.g. BOPerson.list).	
orderBy	Specifies sorting (with the real alias and not %1\$s. in front of the field name) Please note: Do not use if "groupByCols" is applied. This might lead to SQL errors if sorting is based on a field that is not included in the "group by" clause (in case the client has not requested it).	X
groupByCols	Specifies the "group by" fields (in the order) for the SQL statement. The value includes a list of acronyms including their database field (with the real alias and not %1\$s. in front of the field name). The interpreter only adds the fields requested by the client (including their acronyms) to the group by clause. For example: Acronym1=alias.field1 Acronym2=alias.field2	X
filterBy	Specifies fixed filters (with the real alias and not %1\$s. in front of the field name)	X
checkRespAreaMode	Checks the responsibility area of the current user. Modes: "none": no check direct: directly checked by a field of the data source (joined to vab_tab). Use of --DEFAULT-- if empty or null directnotempty: check directly via a field of the data source (joined to vab_tab) "person": check via VAB of person "machine": check via VAB of machine	X

checkRespAreaField	Specifies the field including the responsibility area for direct/directnotempty. Join field if person or machine (with the real alias and not %1\$. in front of the field name)	X
checkRespAreaDefaultValue	Specifies the default value for checking the responsibility area (if the parameter has not been specified by the client). The default value is true. Valid values are true and false.	X
dataTabLabel	Name of the result set. The field normally remains empty and is only required if special processing is performed by further user exits.	X
tableClause	Specifies the table clause without key word FROM. Also the alias must be included. Example: "fmea_eval_nbr_catalog fmeacat".	
tableClauseLongterm	Specifies the table clause for long-term data (only relevant if long-term data exist). If nothing is entered the tableClause is used as tableClauseLongterm.	X
conditionalLongtermKey	This value specifies if the long-term data tables are only conditionally available. The condition as to whether the tables are available is checked by the Configuration Manager. The feature key of the Configuration Manager has to be entered here for checking.	X
mergeOnAttributeLevel	Attribute available as of SP11: This attribute controls how individual DataObjects are merged over several scopes (a single DataObject is identified by the attribute "name"). If the attribute mergeOnAttributeLevel does not exist or the value is not equal "Y", the behavior is the same than before SP11 (backward compatibility). This means that the whole configuration of the DataObjects (not the whole file, but only the entire row) is completely overwritten by a specific scope. For example, if a filterBy is introduced in the custom scope, the complete DataObject in the standard scope is replaced. If the standard is then	

	extended, the standard extension is not applied in the custom scope. If the attribute is set to "Y", the merge behavior changes and one attribute is merged after the other and not the complete DataObject at a time. Refer to the following subchapter for details. (This setting is the default setting for new DataObject configurations as of SP11; existing older configurations are not changed). You can find details and examples in the next subchapter.	
--	--	--

1.5.3.1 Rules for the merge of SQL attributes on attribute level

"Merging by attribute" is available as of Service Pack 11.

Only "name" is a mandatory attribute if you merge DataObjects on attribute level. The other values are acquired from the specific scope.

The following applies for most attributes:

- An **empty attribute** means that the value of the general scope **is used**.
- A **populated attribute** means that the value of the general scope is **overwritten**.

These SQL attributes are an exception:

- tableClauseLongterm
- tableClause
- filterBy
- groupByCols
- orderBy

The following applies with these SQL attributes:

- An **empty attribute** means that the value of the general scope **is used**.
- A **populated attribute** means that the value of the general scope is **written before** it (i.e. the value of the specific scope is written after the value of the general scope). To separate the content of the general scope from the content of the specific scope, different separators are used for the different attributes:
 - o tableClauseLongterm: one space
 - o tableClause: one space

- filterBy: an „AND“; also the general scope is in brackets. . If there is no keyword "\$LOWER_SCOPE_VALUE\$" in the specific scope, then the specific scope is in brackets.
- orderBy: a comma („“)
- groupByCols: a pipe („|“)
- If the attribute contains the keyword \$NO_LOWER_SCOPE\$, the value of the general scope is completely replaced by the value of the attribute from the specific scope.
- If the attribute includes the key word \$LOWER_SCOPE_VALUE\$, the general scope is copied to this position (with the specific separator) exactly.

1.5.3.2 Examples of the merge of SQL attributes on attribute level

"Merging by attribute" is available as of Service Pack 11.

The following examples only refer to the attributes tableClauseLongterm, tableClause, filterBy, orderBy and groupByCols. The other attributes follow a very simple scheme: If the attribute in more specific scope is populated, the general scope is replaced. If attribute is not filled in the specific scope, the attribute from the general scope is used.

In the following, you can find examples of the behavior, if you merge on attribute level (mergeOnAttributeLevel=Y). We used "filterBy" in our examples. The first row is the general scope (e.g. standard), the second row is the specific scope (e.g. custom) and the third row is the result of the merge.

```
"xy.field_x = '42'",
"foo.field_y = '24'",
"(xy.field_x = '42') and (foo.field_y = '24')"
```

```
"xy.field_x = '42'",
"",
"xy.field_x = '42'"
```

```
"",
"foo.field_y = '24'",
"foo.field_y = '24'"
```

```
"xy.field_x = '42'",
"foo.field_y$LANG = 'foo' $LOWER_SCOPE_VALUE$",
"foo.field_y$LANG = 'foo' and (xy.field_x = '42')"
```

```
"xy.field_x = '42'",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
"foo.field_y = '24' and (xy.field_x = '42')"
```

```
"",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
"foo.field_y = '24'"
```

```
"xy.field_x = '42'",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and (xy.field_x = '42') and bar.field_z = 'world'"
```

```
"",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"
```

```

"xy.field_x = '42'",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
"(xy.field_x = '42') and foo.field_y = '24'"

"",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
"foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
"foo.field_y = '24'"

"",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
"foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"xy.field_x = '42'",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
"foo.field_y = '24'"

"",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
"foo.field_y = '24'"

"satz_art = 'U'",
"$NO_LOWER_SCOPE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
"masch_nr = '4711' and (satz_art = 'U')"
```

```

"satz_art = 'U'",
"masch_nr = '4711' $NO_LOWER_SCOPE$LOWER_SCOPE_VALUE$",
"masch_nr = '4711' LOWER_SCOPE_VALUE$" => invalid SQL

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$NO_LOWER_SCOPE$",
"masch_nr = '4711' (satz_art = 'U') andNO_LOWER_SCOPE$" => invalid SQL

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$ $LOWER_SCOPE_VALUE$",
"masch_nr = '4711' (satz_art = 'U') and $LOWER_SCOPE_VALUE$" => invalid SQL

"satz_art = 'U'",
"$LOWER_SCOPE_VALUE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
"(satz_art = 'U') and masch_nr = '4711' $LOWER_SCOPE_VALUE$" => invalid SQL

```

1.6 Exits

Exits provide the entry points to enable changes to the defined behavior by programming.



Instead of the user exits and program exits presented below, use the [GlobalExits](#). The GlobalExits ensure the greatest possible compatibility in the further development of the system, as they are supported equally for all service types.

1.6.1 Available user exits

User exits provide entry points that are not modified by releases. In case of releases, the interfaces are respected and preserved by MPDV in a backwards compatible manner.

1.6.1.1 `sdiInterpretedSqlModifyRequest`

The application developer can change the parameters and the column configurator using this exit. You can also create temporary tables as this exit includes access to the DB session which is also used by the main SQL.

1.6.1.2 `sdiAddResultTransformationCallbacks`

Using this exit, the application developer can modify the service result after having executed the SQL statement. Rows can be deleted, added or changed. The application developer registers a callback to this end, which is called for each row.

1.6.1.3 `sdiInterpretedSqlCleanup`

Using this exit, the application developer can undertake cleanup actions. This exit is always executed, whether or not errors occurred. Here you can e.g. clean up temporary tables as you can access the DB session of the main SQL.

1.6.2 Available program exits

Program exits offer extended functionality that is not backwards compatible. Changes can be made with every update. Users of program exits must test if their modifications still work after an update.

Therefore, program exits are of limited value with modifications.

1.6.2.1 `sdiAugmentSql`

With this user exit, the interpreted Java service provides the application developer with an option to extend the generated SQL shortly before it is executed.

1.6.3 Specifications for the implementation class of the exit

Package name: You must include the class in a package that consists of the domain name (in lower case letters). Further subpackages are **not** allowed.

Example: The service is requested "MDUserAccountRules.list", consequently the package is called "mduseraccountrules"

Class name: The class must have a name of the following structure: domain name in lower case letters, whereas the first letter is written in capital letters, the name of the service function follows and is written in lower case letters, whereas the first letter is once again written in upper case.

Example: The service is requested "MDUserAccountRules.list", consequently the class is called "MduseraccountrulesList"



The following definition applies for customized class names:

Customized names include "_" (see naming conventions)

After "_" the first letter is changed to upper case and the underscore is skipped

Example: U_CUST_Units_sample.list => UCustUnitsSampleList

Implemented interfaces / methods: no specifications

Other: The class must have a default constructor without parameters

Compilation: The Jar files *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar* must be included in the class path for the compile process.

Deployment: The class file of the exit must be stored in the directory
<JDIR>/MOC/<System>/userexit/<scope> including package directory structure.

Example: Exit sdiAugmentSql for service "MDUserAccountRules.list":

```
package mduseraccountrules;
```

```
import de.mpdv.customization.userExit.IUserExitParam;
```

```
/**
```

```
 * Sample user exit
```

```
 *
```

```
 *
```

```
 */
```

```
public class MduseraccountrulesList
{
```

```
    public void sdiAugmentSql(final IUserExitParam param)
```

```
    {
```

```
        // TODO implementation
```

```
    }
```

```
}
```

Directory structure on the server (System 1, scope custom):
<JDIR>/MOC/1/userexit/custom/mduseraccountrules/MduseraccountrulesList.class

1.6.4 Interfaces

1.6.4.1 Class: InterpretedJavaServiceUeContext

de.mpdv.sdi.data.ue.InterpretedJavaServiceUeContext
-hydraNow: Calendar -userId: String
+getHydraNow(): Calendar +getUserId(): String

This context class provides data that is generally useful for interpreted Java services as regards to exits.

Field	Description
hydraNow	The property "hydraNow" is a time stamp created at the beginning of web service processing and, as a result, can be used as reference time stamp for the current web service call.
userId	Includes the user logged on to the client.

1.6.4.2 User exit: sdiInterpretedSqlModifyRequest

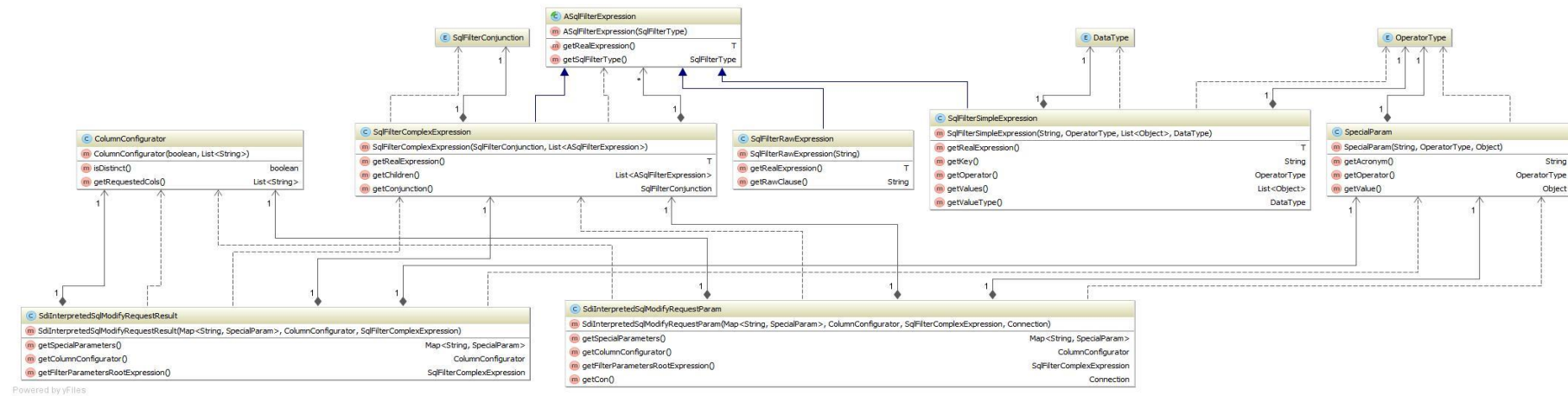
Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiInterpretedSqlModifyRequestParam	Parameter structure for the user exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiInterpretedSqlModifyRequestResult	Result structure for the user exit

Class diagram of parameter and result structures:



Powered by yFiles

1.6.4.3 Class *SdiInterpretedSqlModifyRequestParam*

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"
filterParametersRootExpression	SqlFilterComplexExpression	Root node of the SQL filter tree. All parameters of type "isFilterParameter" are included in this tree.
con	Connection	DB session that is also used by the main SQL.

1.6.4.4 Class *SdiInterpretedSqlModifyRequestResult*

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the modified column configuration
specialParameters	Map<String, SpecialParam>	Includes the modified SpecialParameters
filterParametersRootExpression	SqlFilterComplexExpression	Includes the modified FilterParameter

1.6.4.5 User exit: *sdiAddResultTransformationCallbacks*

Parameter key in IUserExitParam:

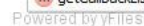
Key name	Type	Description
param	SdiAddResultTransformationCallbackParam	Parameter structure for the

		user exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAddResultTransformationCallbackResult	Result structure for the user exit: Note: If you only want to change the input row, you need not generate a result of the class "SdiModifyResultRowResult" and therefore also the key "result" is not necessary in IUserExitParam. A result of the class "SdiModifyResultRowResult" is generated, if rows are deleted or added.

Class diagram of parameter and result structures:



1.6.4.6 Class SdiAddResultTransformationCallbacksParam

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
dataRowBuilder	ISdiDataRowBuilder	Using this builder, you can generate the instances ISdiDataRow. A direct implementation of ISdiDataRow is not allowed!

1.6.4.7 Class SdiAddResultTransformationCallbacksResult

Field	Type	Description
callbackList	List<ISdiResultTransformationCallback>	Callback list for the result transformation

1.6.4.8 Interface ISdiResultTransformationCallback

Method	Description
transform(ISdiDataRow dataRow, boolean isLastRow): ISdiDataRowStream	Callback method for the result transformation Return type: ISdiDataRowStream must not be NULL: Includes the rows as stream (to support streaming) once the input row has been edited in the user exit. The service then returns the stream rows as result to the client. You can use the class SdiEagerDataRowStream to modify the current row and to return few result rows. If you want to return large amounts of data, you must implement a

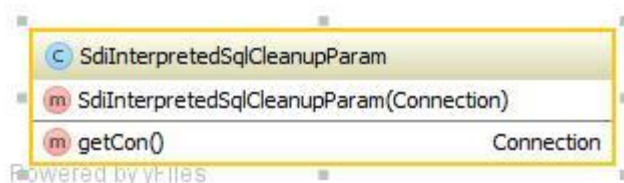
	stream that takes data rows from an external data source. If you must create ISdiDataRow instances, you must use ISdiDataRowFactory from SdiAddResultTransformationCallbacksParam. A direct implementation of ISdiDataRow is not allowed! Input: ISdiDataRow dataRow: Includes a result row that the interpreter creates as service result. isLastRow boolean: TRUE, if this row is the last row, otherwise FALSE
--	--

1.6.4.9 User exit: sdilInterpretedSqlCleanup

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiInterpretedSqlCleanupParam	Parameter structure for the user exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Class diagram of the parameter structure:



1.6.4.10 Class SdiInterpretedSqlCleanupParam

Field	Type	Description
-------	------	-------------

con	Connection	DB session that is also used by the main SQL.
-----	------------	---

1.6.4.11 Program exit: sdiAugmentSql

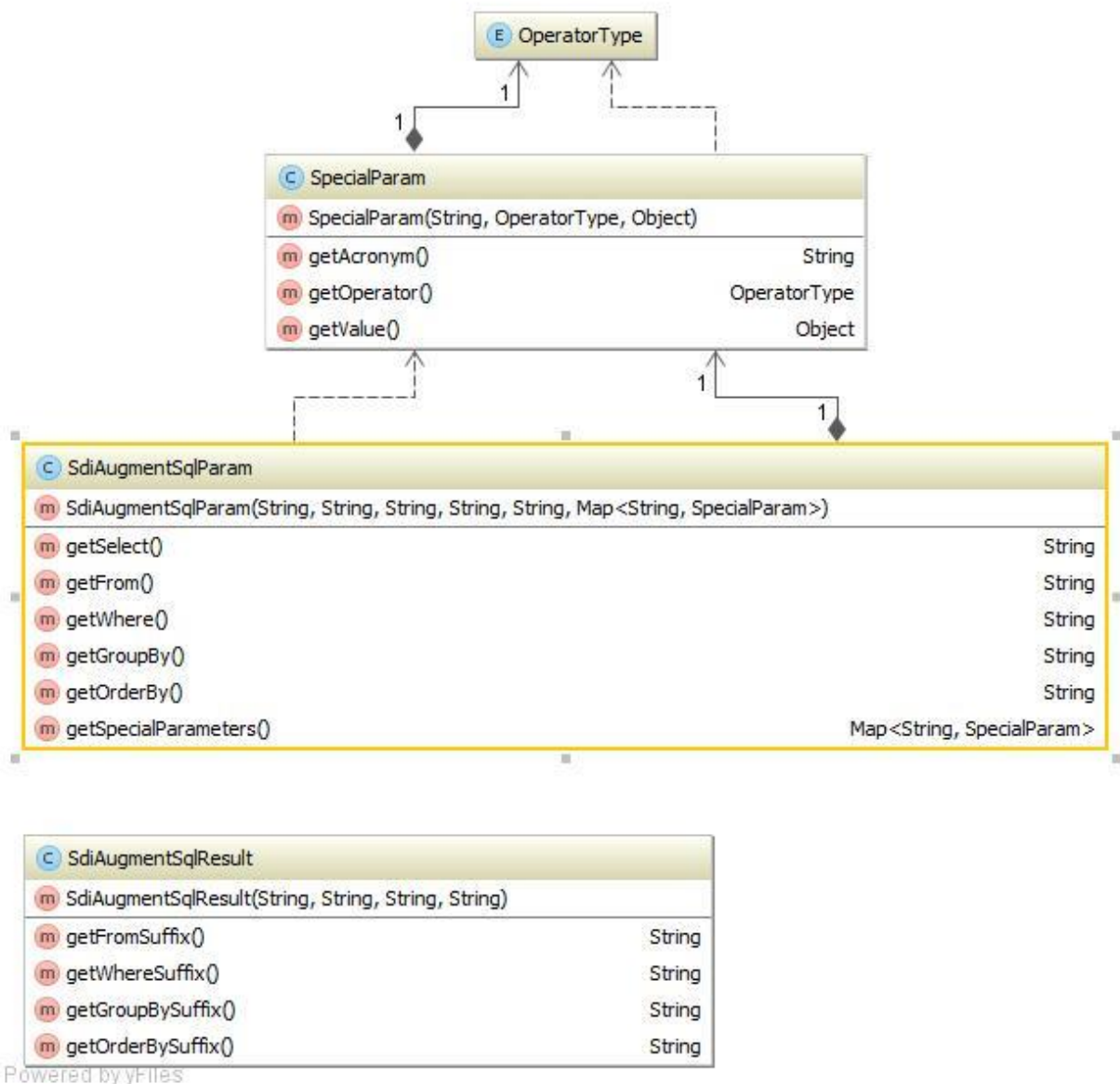
Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAugmentSqlParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAugmentSqlResult	Result structure for the program exit

Class diagram of parameter and result structures:



1.6.4.12 Class SdiAugmentSqlParam

Field	Type	Description
select	String	SELECT clause created based on the configuration (only includes the column part up to FROM, only without the key word SELECT)
from	String	FROM clause created based on the configuration (without the key word FROM)

WHERE	String	WHERE clause created based on the configuration (without the key word WHERE)
groupBy	String	GROUP BY clause created based on the configuration
orderBy	String	ORDER BY clause created based on the configuration
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

1.6.4.13 Class SdiAugmentSqlResult

Field	Type	Description
fromSuffix	String	Suffix for the FROM clause created in the program exit or NULL
whereSuffix	String	Suffix for the WHERE clause created in the program exit or NULL
groupBySuffix	String	Suffix for the GROUP BY clause created in the program exit or NULL
orderBySuffix	String	Suffix for the ORDER BY clause created in the program exit or NULL